

- Description

This module manages an energy consumption monitoring system for factories, ensuring that:

1. **Energy usage is recorded periodically and compared against predefined limits.**
2. **If energy consumption exceeds the limit, an overuse warning is triggered.**
3. **The system recommends energy-saving measures when excessive consumption is detected.**
4. **A log of all energy usage data and alerts is maintained for further analysis.**

This system helps factories optimize energy consumption, reduce costs, and promote sustainability.

- Specification

module FactoryEnergyMonitoring

var: energyUsed, productionOutput, efficiency, maxAllowedUsage: real

var energyLog: sequence of string

process recordUsage(energyUsed: real, productionOutput: real)

functional scenario form:

energyUsed <= maxAllowedUsage && energyLog = energyLog $\oplus$ ("Usage Recorded: " + energyUsed + " kWh"), Energy usage recorded successfully.

||

energyUsed > maxAllowedUsage && energyLog = energyLog $\oplus$ ("Overuse Warning: " + energyUsed + " kWh"), Overuse warning triggered.

end process

end module

- Java Code

```
package code;
import java.io.*;
import java.util.*;
import java.util.concurrent.ThreadLocalRandom;
/**
 * Factory Energy System - Extended
 * - Numerical-condition centric checks
 * - Policy thresholds (max usage, min efficiency, peak-
hour multipliers)
 * - Anomaly detection (negative/zero inputs, extreme ratio, surge)
 * - Rolling stats (mean/median/min/max/p95)
 * - Log & CSV export
 * - Reproducible simulation (seed)
 * - Test toggles: VT / FT / Z3 / UVT
 *
 * Only one public class; other helpers are package-private.
```

```

*/
public class c118_FactoryEnergySystemEx {
    // ===== Toggle switches for test groups =====
    private static final boolean RUN_VT = false;
    private static final boolean RUN_FT = false;
    private static final boolean RUN_Z3 = false;
    private static final boolean RUN_UVT = true;
    // ===== Console verbosity =====
    private static final boolean VERBOSE_CONSOLE = true;
    public static void main(String[] args) {
        EnergyPolicy policy = new EnergyPolicy(
            1000.0, // maxAllowedUsage (kWh)
            0.50, // minEfficiency (units/kWh)
            1.30, // peakHourMultiplier threshold factor
            18, 21 // peak hours [start, end)
        );
        EnergyMonitorEx monitor = new EnergyMonitorEx(policy);
        // Handful quick demos (you can comment them out)
        monitor.recordUsage(800.0, 1200.0, 10); // ok
        monitor.recordUsage(1200.0, 400.0, 19); // overuse in peak
        monitor.recordUsage(-10.0, 200.0, 9); // invalid
        // ===== Execute test groups =====
        // if (RUN_VT) TestSuites.runVT(monitor);
        // if (RUN_FT) TestSuites.runFT(monitor);
        // if (RUN_Z3) TestSuites.runZ3(monitor);
        // if (RUN_UVT) TestSuites.runUVT(monitor);

        TestSuites.runVT(monitor);
        // TestSuites.runFT(monitor);
        // TestSuites.runZ3(monitor);
        // TestSuites.runUVT(monitor);
        // ===== Print and export =====
        monitor.printEnergyLog();
        EnergyStats stats = monitor.computeStats();
        if (VERBOSE_CONSOLE) {
            System.out.println("\n=== Summary Statistics ===");
            System.out.printf(Locale.US, "count=%d, totalEnergy=%.2f kW
h, totalOutput=%.2f units%n",
                stats.count, stats.totalEnergy, stats.totalOutput);
            System.out.printf(Locale.US, "efficiencyMean=%.4f, min=%.4f
, max=%.4f, p95=%.4f%n",
                stats.meanEfficiency, stats.minEfficiency, stats.ma
xEfficiency, stats.p95Efficiency);
        }
    }
}

```

```

        try {
            String csv = "energy_log_c118.csv";
            monitor.exportCsv(csv);
            if (VERBOSE_CONSOLE) {
                System.out.println("CSV exported: " + csv);
            }
        } catch (IOException e) {
            System.out.println("Failed to export CSV: " + e.getMessage(
));
        }
    }
}

// ===== Core Classes =====
/** Policy & thresholds for energy usage. */
class EnergyPolicy {
    final double maxAllowedUsage;    // kWh
    final double minEfficiency;      // units/kWh
    final double peakHourMultiplier; // factor if usage in peak hours
    final int peakStartHour;         // inclusive
    final int peakEndHour;           // exclusive
    EnergyPolicy(double maxAllowedUsage, double minEfficiency,
        double peakHourMultiplier, int peakStartHour, int peak
EndHour) {
        this.maxAllowedUsage = maxAllowedUsage;
        this.minEfficiency = minEfficiency;
        this.peakHourMultiplier = peakHourMultiplier;
        this.peakStartHour = peakStartHour;
        this.peakEndHour = peakEndHour;
    }
    boolean isPeakHour(int hour0to23) {
        if (peakStartHour <= peakEndHour) {
            return hour0to23 >= peakStartHour && hour0to23 < peakEndHour;
        }
        // wrap-around case (not used here)
        return hour0to23 >= peakStartHour || hour0to23 < peakEndHour;
    }
}

/** Single usage record. */
class EnergyUsageRecord {
    final double energyUsed;
    final double productionOutput;
    final double efficiency;    // units/kWh
    final int hourOfDay;        // 0-23

```

```

    final String message;          // formatted line
    final List<String> warnings;    // triggered warnings
    EnergyUsageRecord(double e, double p, int hour, double eff, String
msg, List<String> warns) {
        this.energyUsed = e;
        this.productionOutput = p;
        this.efficiency = eff;
        this.hourOfDay = hour;
        this.message = msg;
        this.warnings = warns;
    }
}

/** Anomaly/Rule detector (pure numerical rules). */
class EnergyAnomalyDetector {
    static List<String> detect(EnergyPolicy policy,
                                double energyUsed, double productionOutp
ut,
                                double efficiency, int hourOfDay,
                                EnergyUsageRecord prev) {
        List<String> warns = new ArrayList<>();
        // 1) Invalid input guards
        if (energyUsed <= 0) warns.add("Invalid: energyUsed <= 0");
        if (productionOutput <= 0) warns.add("Invalid: productionOutput
<= 0");
        // 2) Overuse vs policy
        if (energyUsed > policy.maxAllowedUsage) {
            warns.add("WARNING: Energy overuse (> maxAllowedUsage)");
        }
        // 3) Low efficiency
        if (efficiency < policy.minEfficiency) {
            warns.add("WARNING: Low efficiency (< minEfficiency)");
        }
        // 4) Peak-hour stress (scaled condition)
        if (policy.isPeakHour(hourOfDay) && energyUsed > policy.maxAllo
wedUsage / policy.peakHourMultiplier) {
            warns.add("WARNING: Overuse during peak hour");
        }
        // 5) Extreme ratio (output/energy unusually high)
        if (efficiency > policy.minEfficiency * 200) {
            warns.add("WARNING: Suspiciously high efficiency");
        }
        // 6) Surge detection vs previous record (if available)
        if (prev != null) {

```

```

        if (energyUsed > prev.energyUsed * 2.5) warns.add("Surge: energyUsed spiked > 2.5x");
        if (productionOutput < prev.productionOutput * 0.4) warns.add("Drop: productionOutput fell < 0.4x");
    }
    return warns;
}
}

/** Extended Monitor with stats & persistence. */
class EnergyMonitorEx {
    private final EnergyPolicy policy;
    private final List<EnergyUsageRecord> log = new ArrayList<>();
    EnergyMonitorEx(EnergyPolicy policy) {
        this.policy = policy;
    }

    /** Core API - records a usage with hour-of-day context. */
    // public void recordUsage(double energyUsed, double productionOutput
    // , int hourOfDay) {
    //     if (hourOfDay < 0 || hourOfDay > 23) {
    //         System.out.println("Invalid hour: must be 0-23");
    //         return;
    //     }
    //
    //     // Pre-check for zero/negative to avoid divide-by-zero
    //     if (energyUsed <= 0 || productionOutput <= 0) {
    //         String msg = "Invalid input: Energy usage and production output must be greater than zero.";
    //         System.out.println(msg);
    //         Log.add(new EnergyUsageRecord(energyUsed, productionOutput,
    //             hourOfDay, Double.NaN, msg,
    //                 Arrays.asList("Invalid input")));
    //         return;
    //     }
    //
    //     double efficiency = productionOutput / energyUsed;
    //
    //     EnergyUsageRecord prev = log.isEmpty() ? null : log.get(log.size() - 1);
    //     List<String> warns = EnergyAnomalyDetector.detect(policy, energyUsed, productionOutput, efficiency, hourOfDay, prev);
    //
    //     String base = String.format(Locale.US,
    //         "Hour %02d | Energy Used: %.2f kWh | Production Output: %.2f units | Efficiency: %.4f units/kWh",

```

```

//          hourOfDay, energyUsed, productionOutput, efficiency));
//
//      StringBuilder sb = new StringBuilder(base);
//      if (!warns.isEmpty()) {
//          sb.append(" | ");
//          for (int i = 0; i < warns.size(); i++) {
//              if (i > 0) sb.append("; ");
//              sb.append(warns.get(i));
//          }
//      }
//
//      String line = sb.toString();
//      System.out.println(line);
//      Log.add(new EnergyUsageRecord(energyUsed, productionOutput, ho
urOfDay, efficiency, line, warns));
//  }
//
//  /** Convenience wrapper without hour -> assign random hour. */
//  public void recordUsage(double energyUsed, double productionOutput
) {
//      int hour = ThreadLocalRandom.current().nextInt(0, 24);
//      recordUsage(energyUsed, productionOutput, hour);
//  }
//
//  public void printEnergyLog() {
//      System.out.println("\nEnergy Consumption Log:");
//      for (EnergyUsageRecord r : Log) {
//          System.out.println(r.message);
//      }
//  }
//
//  public EnergyStats computeStats() {
//      return EnergyStats.from(Log);
//  }
//
//  public void exportCsv(String filePath) throws IOException {
//      try (PrintWriter pw = new PrintWriter(new BufferedWriter(new F
ileWriter(filePath)))) {
//          pw.println("hour,energy_kwh,output_units,efficiency,has_war
ning,warnings");
//          for (EnergyUsageRecord r : Log) {
//              String warn = String.join("|", r.warnings);
//              pw.printf(Locale.US, "%d,%.6f,%.6f,%.6f,%b,%s\n",
//                  r.hourOfDay, r.energyUsed, r.productionOutput,

```

```

//                                r.efficiency, !r.warnings.isEmpty(), escapeCsv(w
arn));
//                                }
//                                }
//                                }
//
//    private static String escapeCsv(String s) {
//        if (s == null) return "";
//        if (s.contains(",") || s.contains("\"")) {
//            return "\"" + s.replace("\"", "\\\"") + "\"";
//        }
//        return s;
//    }
    public void recordUsage(double energyUsed, double productionOutput,
int hourOfDay) {
        if (hourOfDay < 0 || hourOfDay > 23) {
            System.out.println("Invalid hour: must be 0-23");
            return;
        }
        // Pre-check for zero/negative to avoid divide-by-zero
        if (energyUsed <= 0 || productionOutput <= 0) {
            String msg = "Invalid input: Energy usage and production ou
tput must be greater than zero.";
            System.out.println(msg);
            log.add(new EnergyUsageRecord(energyUsed, productionOutput,
hourOfDay, Double.NaN, msg,
                Arrays.asList("Invalid input")));
            return;
        }
        double efficiency = productionOutput / energyUsed;
        EnergyUsageRecord prev = log.isEmpty() ? null : log.get(log.siz
e() - 1);
        List<String> warns = EnergyAnomalyDetector.detect(policy, energ
yUsed, productionOutput, efficiency, hourOfDay, prev);
        String base = String.format(Locale.US,
            "Hour %02d | Energy Used: %.2f kWh | Production Output:
%.2f units | Efficiency: %.4f units/kWh",
            hourOfDay, energyUsed, productionOutput, efficiency);
        StringBuilder sb = new StringBuilder(base);
        if (!warns.isEmpty()) {
            sb.append(" | ");
            for (int i = 0; i < warns.size(); i++) {
                if (i > 0) sb.append("; ");
                sb.append(warns.get(i));
            }
        }
    }
}

```

```

        }
    }
    String line = sb.toString();
    System.out.println(line);
    log.add(new EnergyUsageRecord(energyUsed, productionOutput, hourOfDay, efficiency, line, warns));
}
/** Convenience wrapper without hour -> assign random hour. */
public void recordUsage(double energyUsed, double productionOutput)
{
    int hour = ThreadLocalRandom.current().nextInt(0, 24);
    recordUsage(energyUsed, productionOutput, hour);
}
public void printEnergyLog() {
    System.out.println("\nEnergy Consumption Log:");
    for (EnergyUsageRecord r : log) {
        System.out.println(r.message);
    }
}
public EnergyStats computeStats() {
    return EnergyStats.from(log);
}
public void exportCsv(String filePath) throws IOException {
    try (PrintWriter pw = new PrintWriter(new BufferedWriter(new FileWriter(filePath)))) {
        pw.println("hour,energy_kwh,output_units,efficiency,has_warning,warnings");
        for (EnergyUsageRecord r : log) {
            String warn = String.join("|", r.warnings);
            pw.printf(Locale.US, "%d,%.6f,%.6f,%.6f,%b,%s\n",
                r.hourOfDay, r.energyUsed, r.productionOutput,
                r.efficiency, !r.warnings.isEmpty(), escapeCsv(warn));
        }
    }
}
private static String escapeCsv(String s) {
    if (s == null) return "";
    if (s.contains(",") || s.contains("\"")) {
        return "\"" + s.replace("\"", "\\\"") + "\"";
    }
    return s;
}
}

```

```

/** Summary statistics for efficiency & totals. */
class EnergyStats {
    final int count;
    final double totalEnergy;
    final double totalOutput;
    final double meanEfficiency;
    final double minEfficiency;
    final double maxEfficiency;
    final double p95Efficiency;
    EnergyStats(int count, double totalEnergy, double totalOutput,
        double meanEfficiency, double minEfficiency, double max
Efficiency, double p95Efficiency) {
        this.count = count;
        this.totalEnergy = totalEnergy;
        this.totalOutput = totalOutput;
        this.meanEfficiency = meanEfficiency;
        this.minEfficiency = minEfficiency;
        this.maxEfficiency = maxEfficiency;
        this.p95Efficiency = p95Efficiency;
    }
    static EnergyStats from(List<EnergyUsageRecord> list) {
        if (list.isEmpty()) {
            return new EnergyStats(0, 0, 0, Double.NaN, Double.NaN, Dou
ble.NaN, Double.NaN);
        }
        List<Double> effs = new ArrayList<>();
        double eSum = 0, pSum = 0;
        double min = Double.POSITIVE_INFINITY, max = Double.NEGATIVE_IN
FINITY;
        for (EnergyUsageRecord r : list) {
            if (!Double.isFinite(r.efficiency)) continue;
            effs.add(r.efficiency);
            eSum += r.energyUsed;
            pSum += r.productionOutput;
            min = Math.min(min, r.efficiency);
            max = Math.max(max, r.efficiency);
        }
        if (effs.isEmpty()) {
            return new EnergyStats(list.size(), eSum, pSum, Double.NaN,
Double.NaN, Double.NaN, Double.NaN);
        }
        double mean = 0;
        for (double v : effs) mean += v;
        mean /= effs.size();
    }
}

```

```

        Collections.sort(effs);
        double p95 = effs.get((int)Math.floor(0.95 * (effs.size()-1)));
        return new EnergyStats(list.size(), eSum, pSum, mean, min, max,
p95);
    }
}

// ===== Simulation & Test Suites =====
/** Deterministic simulator (seeded) to produce synthetic data if needed. */
class EnergySimulator {
    private final Random rnd;
    EnergySimulator(long seed) {
        rnd = new Random(seed);
    }
    /** Generate a plausible usage and output pair. */
    double[] nextSample() {
        // energy between 100 and 1500
        double energy = 100 + rnd.nextDouble() * 1400;
        // efficiency target around 1.2 ± 0.6
        double eff = 1.2 + rnd.nextGaussian() * 0.6;
        eff = Math.max(0.05, eff); // keep positive
        double output = energy * eff;
        return new double[]{energy, output};
    }
}

/** Centralized test suites with original user-
provided payloads grouped. */
class TestSuites {
    /** Helper to run a matrix of pairs with auto-assigned hours. */
    private static void runPairs(EnergyMonitorEx m, double[][] pairs, i
nt startHour) {
        int hour = startHour;
        for (double[] p : pairs) {
            m.recordUsage(p[0], p[1], hour);
            hour = (hour + 1) % 24;
        }
    }
}

// ----- VT -----
static void runVT(EnergyMonitorEx m) {
    double[][] vt = new double[][]{
        {-929.93, 3398.41},
        {4728.84, 1585.89},
        {3544.78, 437.15},
        {-775.56, 4326.12},
    }
}

```

{3301.20,-4436.36},
{-2355.64,-577.59},
{2069.94,-4698.54},
{-2554.84,-3989.47},
{1411.68,-2635.97},
{-1129.01,-4662.37},
{-705.61,-2685.10},
{-3679.25,-187.01},
{2869.89,-3314.98},
{19.73,-2089.05},
{1235.30,-4042.63},
{-1652.31,-3768.61},
{-881.17,-2702.65},
{-230.48,-1781.02},
{4113.51,-3454.50},
{1239.88,114.39},
{-190.10,-4404.27},
{-4781.30,4745.49},
{4871.58,766.79},
{2752.15,-1090.90},
{3914.09,765.53},
{1184.62,1438.07},
{-3366.22,2440.76},
{3798.39,-4278.28},
{-4679.81,1836.31},
{4136.33,3960.34},
{4828.81,1223.53},
{4621.52,-4262.01},
{-569.94,-4058.13},
{4432.12,-622.88},
{-3357.51,1324.03},
{-1072.24,3660.60},
{2835.98,3888.96},
{409.32,-4106.45},
{-2767.75,-2092.41},
{-1276.05,4046.21},
{-544.76,-923.40},
{-366.00,-1048.28},
{311.41,-2294.98},
{1485.61,3736.57},
{-498.23,-3251.17},
{1857.71,4981.00},
{210.36,-815.44},
{-4495.43,-873.31},

{-4896.96,1203.56},
{1995.52,-3612.68},
{-807.24,2171.48},
{-3637.68,1917.63},
{1668.89,27.42},
{-853.00,-3740.07},
{-565.00,-3434.03},
{-236.86,3073.92},
{4843.32,-2867.20},
{-1657.20,-478.22},
{-180.38,4299.27},
{3022.87,-2257.78},
{1037.47,4722.22},
{-4761.60,1768.89},
{-1600.59,-4053.44},
{1520.17,-693.26},
{-4841.63,-2334.68},
{-2246.37,-2663.87},
{4190.97,-1164.84},
{-3840.32,-3770.34},
{-855.00,-3733.84},
{-656.29,-3267.95},
{4850.67,-4241.27},
{434.27,-3540.24},
{-4496.82,-1401.37},
{4642.46,-2261.43},
{3182.98,4467.60},
{-4446.87,3566.73},
{2855.02,-4244.03},
{3738.28,-1626.75},
{-3643.85,2192.73},
{1042.04,839.64},
{-2411.31,-2656.35},
{-798.40,-1700.44},
{4466.82,-235.36},
{-3776.09,4518.89},
{-2793.75,2146.98},
{2775.04,3665.36},
{-2308.08,3525.30},
{-583.75,-4119.68},
{-3712.25,1898.60},
{-2465.87,-1267.21},
{-2337.14,-4422.94},
{-4468.88,-4616.22},

[illegible]

[illegible] $\{-$ [illegible] $\{-$ [illegible] $\{0.00, 0.00\},$

{106223927242599800}

[illegible]

{10622392272425998000}

[illegible]

{17976931348623157000}

[illegible][illegible]

{17936882642920963000}

[illegible] $\{0.00, 0.00\},$ $\{0.00, 0.00\},$

[illegible]

[illegible][illegible][illegible][illegible][illegible][illegible][illegible][illegible][illegible][illegible][illegible]

[illegible] $\{-$ [illegible] $\{0.00, 0.00\},$ $\{0.00, 0.00\},$ $\{0.00, 0.00\},$

```
{7176448339377356000000000000000000000000000000000000000}
```

00
00
00
000 .00 , -

[illegible] $\{0.00, 0.00\},$ $\{-$ [illegible] $\{-$ [illegible] $\{-$ [illegible]

{17976931348623157000}

[illegible]

[illegible]

[illegible] $\{0.00, 0.00\},$ $\{0.00, 0.00\},$ $\{0.00, 0.00\},$ $\{0.00, 0.00\},$ $\{-$ [illegible]

{779625120911999800}

[illegible] $\{-$ [illegible] $\{-$ [illegible] $\{-$ [illegible] $\{0.00, 0.00\},$ $\{0.00, 0.00\},$ $\{0.00, 0.00\},$ $\{0.00, 0.00\},$ $\{0.00, 0.00\},$ $\{-$ [illegible]

[illegible]

{-1652.31,-3768.61},
{-881.17,-2702.65},
{-230.48,-1781.02},
{4113.51,-3454.50},
{1239.88,114.39},
{-190.10,-4404.27},
{-4781.30,4745.49},
{4871.58,766.79},
{2752.15,-1090.90},
{3914.09,765.53},
{1184.62,1438.07},
{-3366.22,2440.76},
{3798.39,-4278.28},
{-4679.81,1836.31},
{4136.33,3960.34},
{4828.81,1223.53},
{4621.52,-4262.01},
{-569.94,-4058.13},
{4432.12,-622.88},
{-3357.51,1324.03},
{-1072.24,3660.60},
{2835.98,3888.96},
{409.32,-4106.45},
{-2767.75,-2092.41},
{-1276.05,4046.21},
{-544.76,-923.40},
{-366.00,-1048.28},
{311.41,-2294.98},
{1485.61,3736.57},
{-498.23,-3251.17},
{1857.71,4981.00},
{210.36,-815.44},
{-4495.43,-873.31},
{-4896.96,1203.56},
{1995.52,-3612.68},
{-807.24,2171.48},
{-3637.68,1917.63},
{1668.89,27.42},
{-853.00,-3740.07},
{-565.00,-3434.03},
{-236.86,3073.92},
{4843.32,-2867.20},
{-1657.20,-478.22},
{-180.38,4299.27},

```
{3022.87,-2257.78},
{1037.47,4722.22},
{-4761.60,1768.89},
{-1600.59,-4053.44},
{1520.17,-693.26},
{-4841.63,-2334.68},
{-2246.37,-2663.87},
{4190.97,-1164.84},
{-3840.32,-3770.34},
{-855.00,-3733.84},
{-656.29,-3267.95},
{4850.67,-4241.27},
{434.27,-3540.24},
{-4496.82,-1401.37},
{4642.46,-2261.43},
{3182.98,4467.60},
{-4446.87,3566.73},
{2855.02,-4244.03},
{3738.28,-1626.75},
{-3643.85,2192.73},
{1042.04,839.64},
{-2411.31,-2656.35},
{-798.40,-1700.44},
{4466.82,-235.36},
{-3776.09,4518.89},
{-2793.75,2146.98},
{2775.04,3665.36},
{-2308.08,3525.30},
{-583.75,-4119.68},
{-3712.25,1898.60},
{-2465.87,-1267.21},
{-2337.14,-4422.94},
{-4468.88,-4616.22},
{-4653.17,4986.41},
{1420.60,-3969.65},
{3950.26,-236.26},
{4424.71,-4843.33},
{-791.76,-4290.14},
{463.87,-2862.77},
{-2781.18,3135.68},
{-2126.09,-4837.62},
};
runPairs(m, z3, 18);
}
```

```
// ----- UVT -----
```

```
static void runUVT(EnergyMonitorEx m) {
```

```
    double[][] uvt = new double[][]{
```

```
        {-929.93,3398.41},
        {4728.84,1585.89},
        {3544.78,437.15},
        {-775.56,4326.12},
        {3301.20,-4436.36},
        {-2355.64,-577.59},
        {2069.94,-4698.54},
        {-2554.84,-3989.47},
        {1411.68,-2635.97},
        {-1129.01,-4662.37},
        {-705.61,-2685.10},
        {-3679.25,-187.01},
        {2869.89,-3314.98},
        {19.73,-2089.05},
        {1235.30,-4042.63},
        {-1652.31,-3768.61},
        {-881.17,-2702.65},
        {-230.48,-1781.02},
        {4113.51,-3454.50},
        {1239.88,114.39},
        {-190.10,-4404.27},
        {-4781.30,4745.49},
        {4871.58,766.79},
        {2752.15,-1090.90},
        {3914.09,765.53},
        {1184.62,1438.07},
        {-3366.22,2440.76},
        {3798.39,-4278.28},
        {-4679.81,1836.31},
        {4136.33,3960.34},
        {4828.81,1223.53},
        {4621.52,-4262.01},
        {-569.94,-4058.13},
        {4432.12,-622.88},
        {-3357.51,1324.03},
        {-1072.24,3660.60},
        {2835.98,3888.96},
        {409.32,-4106.45},
        {-2767.75,-2092.41},
        {-1276.05,4046.21},
        {-544.76,-923.40},
```

{-366.00,-1048.28},
{311.41,-2294.98},
{1485.61,3736.57},
{-498.23,-3251.17},
{1857.71,4981.00},
{210.36,-815.44},
{-4495.43,-873.31},
{-4896.96,1203.56},
{1995.52,-3612.68},
{-807.24,2171.48},
{-3637.68,1917.63},
{1668.89,27.42},
{-853.00,-3740.07},
{-565.00,-3434.03},
{-236.86,3073.92},
{4843.32,-2867.20},
{-1657.20,-478.22},
{-180.38,4299.27},
{3022.87,-2257.78},
{1037.47,4722.22},
{-4761.60,1768.89},
{-1600.59,-4053.44},
{1520.17,-693.26},
{-4841.63,-2334.68},
{-2246.37,-2663.87},
{4190.97,-1164.84},
{-3840.32,-3770.34},
{-855.00,-3733.84},
{-656.29,-3267.95},
{4850.67,-4241.27},
{434.27,-3540.24},
{-4496.82,-1401.37},
{4642.46,-2261.43},
{3182.98,4467.60},
{-4446.87,3566.73},
{2855.02,-4244.03},
{3738.28,-1626.75},
{-3643.85,2192.73},
{1042.04,839.64},
{-2411.31,-2656.35},
{-798.40,-1700.44},
{4466.82,-235.36},
{-3776.09,4518.89},
{-2793.75,2146.98},

```

        {2775.04,3665.36},
        {-2308.08,3525.30},
        {-583.75,-4119.68},
        {-3712.25,1898.60},
        {-2465.87,-1267.21},
        {-2337.14,-4422.94},
        {-4468.88,-4616.22},
        {-4653.17,4986.41},
        {1420.60,-3969.65},
        {3950.26,-236.26},
        {4424.71,-4843.33},
        {-791.76,-4290.14},
        {463.87,-2862.77},
        {-2781.18,3135.68},
        {-2126.09,-4837.62},
    };
    runPairs(m, uvt, 5);
}
}

```

- Coverage report

1. Test case number: 100
2. V-Method testing branch coverage: 82.1%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ c118_FactoryEnergySystemEx.java	 75.7 %	53	17	70
> EnergyPolicy	 40.0 %	4	6	10
> EnergyAnomalyDetector	 80.0 %	16	4	20
> EnergyStats	 80.0 %	8	2	10
▼ EnergyMonitorEx	 82.1 %	23	5	28
escapeCsv(String)	 50.0 %	3	3	6
recordUsage(double, double, int)	 87.5 %	14	2	16
exportCsv(String)	 100.0 %	4	0	4
printEnergyLog()	 100.0 %	2	0	2
EnergyMonitorEx(EnergyPolicy)		0	0	0
computeStats()		0	0	0
> TestSuites	 100.0 %	2	0	2
▼ c118_FactoryEnergySystemEx		0	0	0
main(String[])		0	0	0
▼ EnergyUsageRecord		0	0	0
EnergyUsageRecord(double, double,		0	0	0

3. Vibration testing instruction coverage: 92.3%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ c118_FactoryEnergySystemEx.java	 33.0 %	1,995	4,053	6,048
> TestSuites	 25.5 %	1,342	3,927	5,269
> EnergyPolicy	 74.5 %	35	12	47
> EnergyAnomalyDetector	 84.0 %	79	15	94
> EnergyStats	 85.5 %	136	23	159
▼ c118_FactoryEnergySystemEx	 92.2 %	106	9	115
main(String[])	 94.6 %	106	6	112
▼ EnergyMonitorEx	 92.3 %	276	23	299
escapeCsv(String)	 60.0 %	12	8	20
recordUsage(double, double, int)	 97.3 %	146	4	150
EnergyMonitorEx(EnergyPolicy)	 100.0 %	11	0	11
computeStats()	 100.0 %	4	0	4
exportCsv(String)	 100.0 %	83	0	83
printEnergyLog()	 100.0 %	20	0	20
▼ EnergyUsageRecord	 100.0 %	21	0	21
EnergyUsageRecord(double, double,	 100.0 %	21	0	21

4. Fuzz testing branch coverage: 78.6%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ c118_FactoryEnergySystemEx.java	71.4 %	50	20	70
> EnergyPolicy	40.0 %	4	6	10
> EnergyAnomalyDetector	70.0 %	14	6	20
▼ EnergyMonitorEx	78.6 %	22	6	28
escapeCsv(String)	50.0 %	3	3	6
recordUsage(double, double, int)	81.2 %	13	3	16
exportCsv(String)	100.0 %	4	0	4
printEnergyLog()	100.0 %	2	0	2
EnergyMonitorEx(EnergyPolicy)		0	0	0
computeStats()		0	0	0
> EnergyStats	80.0 %	8	2	10
> TestSuites	100.0 %	2	0	2
> c118_FactoryEnergySystemEx		0	0	0
> EnergyUsageRecord		0	0	0

5. Fuzz testing instruction coverage: 92.3%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ c118_FactoryEnergySystemEx.java	25.3 %	730	2,159	2,889
> TestSuites	3.8 %	81	2,029	2,110
> EnergyPolicy	74.5 %	35	12	47
> EnergyAnomalyDetector	79.8 %	75	19	94
> EnergyStats	85.5 %	136	23	159
> c118_FactoryEnergySystemEx	92.2 %	106	9	115
▼ EnergyMonitorEx	92.3 %	276	23	299
escapeCsv(String)	60.0 %	12	8	20
recordUsage(double, double, int)	97.3 %	146	4	150
EnergyMonitorEx(EnergyPolicy)	100.0 %	11	0	11
computeStats()	100.0 %	4	0	4
exportCsv(String)	100.0 %	83	0	83
printEnergyLog()	100.0 %	20	0	20
> EnergyUsageRecord	100.0 %	21	0	21

6. Z3 branch coverage: 82.1%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ c118_FactoryEnergySystemEx.java	75.7 %	53	17	70
> EnergyPolicy	40.0 %	4	6	10
> EnergyAnomalyDetector	80.0 %	16	4	20
> EnergyStats	80.0 %	8	2	10
▼ EnergyMonitorEx	82.1 %	23	5	28
escapeCsv(String)	50.0 %	3	3	6
recordUsage(double, double, int)	87.5 %	14	2	16
exportCsv(String)	100.0 %	4	0	4
printEnergyLog()	100.0 %	2	0	2
EnergyMonitorEx(EnergyPolicy)		0	0	0
computeStats()		0	0	0
> TestSuites	100.0 %	2	0	2
> c118_FactoryEnergySystemEx		0	0	0
> EnergyUsageRecord		0	0	0

7. Z3 instruction coverage: 92.3%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ c118_FactoryEnergySystemEx.java	37.6 %	1,085	1,804	2,889
> TestSuites	20.5 %	432	1,678	2,110
> EnergyPolicy	74.5 %	35	12	47
> EnergyAnomalyDetector	84.0 %	79	15	94
> EnergyStats	85.5 %	136	23	159
> c118_FactoryEnergySystemEx	92.2 %	106	9	115
▼ EnergyMonitorEx	92.3 %	276	23	299
escapeCsv(String)	60.0 %	12	8	20
recordUsage(double, double, int)	97.3 %	146	4	150
EnergyMonitorEx(EnergyPolicy)	100.0 %	11	0	11
computeStats()	100.0 %	4	0	4
exportCsv(String)	100.0 %	83	0	83
printEnergyLog()	100.0 %	20	0	20
> EnergyUsageRecord	100.0 %	21	0	21

8. Unimproved V-method branch coverage: 82.1%

Element		Coverage	Covered Branches	Missed Branches	Total Branches
▼ c118_FactoryEnergySystemEx.java		75.7 %	53	17	70
> EnergyPolicy		40.0 %	4	6	10
> EnergyAnomalyDetector		80.0 %	16	4	20
> EnergyStats		80.0 %	8	2	10
▼ EnergyMonitorEx		82.1 %	23	5	28
escapeCsv(String)		50.0 %	3	3	6
recordUsage(double, double, int)		87.5 %	14	2	16
exportCsv(String)		100.0 %	4	0	4
printEnergyLog()		100.0 %	2	0	2
EnergyMonitorEx(EnergyPolicy)			0	0	0
computeStats()			0	0	0
> TestSuites		100.0 %	2	0	2
> c118_FactoryEnergySystemEx			0	0	0
> EnergyUsageRecord			0	0	0

9. Unimproved V-method instruction coverage: 92.3%

Element		Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ c118_FactoryEnergySystemEx.java		33.0 %	1,995	4,053	6,048
> TestSuites		25.5 %	1,342	3,927	5,269
> EnergyPolicy		74.5 %	35	12	47
> EnergyAnomalyDetector		84.0 %	79	15	94
> EnergyStats		85.5 %	136	23	159
▼ c118_FactoryEnergySystemEx		92.2 %	106	9	115
main(String[])		94.6 %	106	6	112
▼ EnergyMonitorEx		92.3 %	276	23	299
escapeCsv(String)		60.0 %	12	8	20
recordUsage(double, double, int)		97.3 %	146	4	150
EnergyMonitorEx(EnergyPolicy)		100.0 %	11	0	11
computeStats()		100.0 %	4	0	4
exportCsv(String)		100.0 %	83	0	83
printEnergyLog()		100.0 %	20	0	20
▼ EnergyUsageRecord		100.0 %	21	0	21
EnergyUsageRecord(double, double,		100.0 %	21	0	21